

PROJECT 3 2021: GROUP 02

TALENT DIRECTORY APPLICATION

Melisa Bhixa, Bukasa Plamedie Bitota, Ruphin Bolonda,
Mogamat Taahir Boltman, Melven Johannes Booysen, Adriaan Burger

Department of Information Technology
Cape peninsula University of Technology
Cape Town, South Africa

217131085@mycput.ac.za , 219260532@mycput.ac.za, 218321392@mycput.ac.za, 218022972@mycput.ac.za,
219201277@mycput.ac.za, 219014868@mycput.ac.za

Abstract—For the purpose of Project3, Group 2 has set out to implement a Talent Directory Application – A program designed to help match students with relevant student-tutors. As some students would be in need to upskill and others would want to help by offering their talents and skillset in exchange for possible compensation, this system would help solve some of the learning challenges faced in recent times at Cape Peninsula University of Technology (CPUT). This paper outlines the aims, objectives, architecture, implementation and testing methods as it relates to the Talent Directory Application (TDA).

Keywords—*project3, matching system, student-tutor, web-application*

1. INTRODUCTION

The Talent Directory Application will match higher skilled students with students who want to improve their skills in programming/networking or multimedia. Optional monetary incentive is added as an additional feature to this application (students can pay their tutors if they wish to do so).

The Goals and objectives are the following:

The first goal is to enhance a student's technical skills.

The objectives for this goal would be to create a web-based application that connects skilled students in programming, networking, and multimedia with others in need to reinforce their skills. The steps involved in this would include input, process, and output:

Input -> The skilled student uploads their CV and portfolio. The student who wants to upskill chooses the skill name from options on the application.

Process -> The talent directory application processes the inputs and matches user profiles with each other if the skills correspond.

Output -> Once a student is logged in, the recommendation component within the application displays a list of relevant tutor profiles that match the student's needs as defined on their profile. The student also has the option of doing a manual search to find the relevant tutor.

The second goal is to provide technical assessments.

Objectives for this goal include:

Create technical quizzes and tests to assess a student's technical skills. The skilled student would set the assessments up and the student who wants to upscale will complete it. The scores of these tests would help to show areas in need of improvement. Furthermore, the assessments provide valuable practice for the student who is upscaling their skill.

The third and extra goal is to create an additional source of income for students.

Objectives for this goal include:

Allow those who want to earn an extra income to collect payment via the application in exchange for their service. Students can also privately provide their EFT (Electronic Fund Transfer) details – this acts as a failsafe for this function of the application.

2. BACKGROUND

The following problem has been identified: the lack of support classes at CPUT (Cape Peninsula University of Technology) where IT (Information Technology) students can strengthen their practical skills in programming, networking, or multimedia. This is a critical issue that affects many students during their academic journey, and which has become more pronounced during this pandemic with remote learning. Based on concerns raised by first- and second-year students over the past year outlined/quantified by the anonymous student survey¹, many students are not confident in their technical skills when asked to implement projects outside of what they have learnt in class. This problem limits competitiveness and prevents many students from finding a student job in IT companies. It also affects their ability to earn additional income before their graduation.

It is for that reason that it has been decided to create a talent directory application that will allow students who have a solid foundation in the above subjects to showcase their talents or CVs, connecting them with other students who need to boost their skills in these subjects to overcome this problem.

3. RELATED WORK

Most of the Talent Directory Applications for students that we have studied are designed for the purpose of listing University Alumni. The TDAs that offer a matching system are most of the time general-purpose systems that connect job seekers with recruiters.

The system we have implemented is a single-purpose application where all the implemented features have been designed for university and college students.

¹ An example screenshot of this survey can be found in the References section of this paper

Features within the application have been minimized to avoid drowning the student in an ocean of functionalities as it within general-purpose applications.

The system is also scalable since the flow of information remains adaptable to all situations where the matching of talents in a student context is sought.

4. SYSTEM ARCHITECTURE

The focus of this section discusses the architecture artifacts of our proposed solution, which represents the whole Talent Directory Application system. This will include the components listed below. In essence, the following architecture artifacts will be seen as a template for other users to interpret and implement the solution in any work environment of their choice.

4.1 Registration component

4.2 Profile component

4.3 Search component

4.4 Bookmark component

4.5 Recommendation component

4.6 Payment component

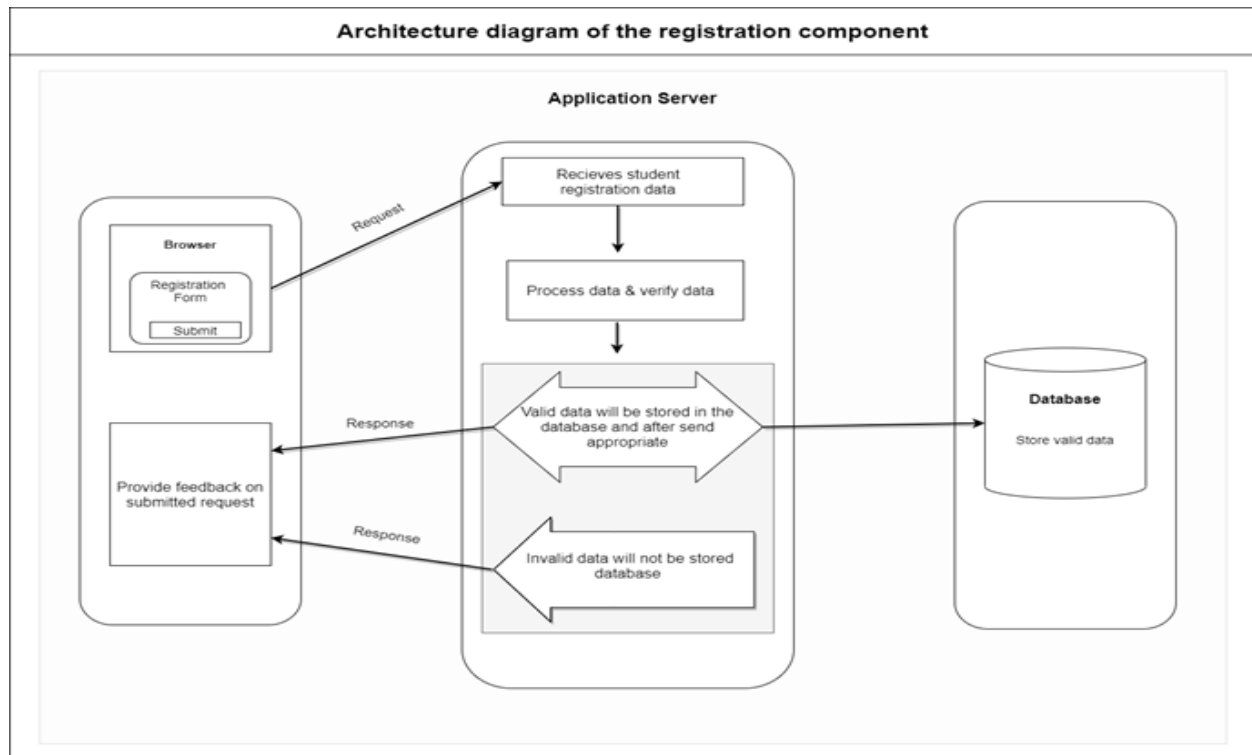
4.7 Self-Assessment component

4.8 Notification component

4.9 Tutor Rating Component

❖ Registration component

The purpose of this part of the application is to register the students to the talent directory application by capturing their details like their name, student email, phone number, and password. These details will be stored in the database to verify next time whether they are a registered member of the application once they try to login to the application again. Students will have the option to register as a student looking to upskill their technical skills or as a student tutor. One of the constraints is that only valid email addresses can be entered and saved. Once they have login their details will be displayed at the top to ensure that they are logged onto their own account.



The application server provides business logic to application programs and the MVC divides the business logic and presentation layer from each other allow them to handle certain tasks to reach the end goal. Therefore, the controller, model and view will manipulate the data from the user and only store valid data into the database. Whether its valid or not valid data, an appropriate response will be displayed to the user's browser.

Firstly, students will be provided with a registration form on their browser where they can fill in the necessary data and submit it. The controller block will receive the data, process the data, and verify if it is valid. Invalid data will not be stored in the database an appropriate message will be displayed. Valid data will be stored into the database (model) and a message will be displayed to the student as feedback on their submission. The message display will be done on the view block.

❖ Profile component

The purpose is to give the student and tutor a place to save their profile information such as name, profile picture and to also list their skillset in the case of tutors or the skills they want to learn for students. The information is then saved and retrieved when the user needs to update skills or change any profile information. The conceptual view for the profile component of tutors and students: (ST = Student Tutor | SU = Student Upskill)

GUI Form_ST	GUI Form_SU
First Name, Last Name Tutor Description	First Name, Last Name
"I want to teach:" Skill Name (dropdown list)	"I want to learn:" Skill Name (dropdown list)
Skill level: (Student, intermediate, professional)	Times available for Upskill: Weekends, Weekdays, After 17h, Before 17h (Checkbox)
Available Teaching Times: Weekends, Weekdays, After 17h, Before 17h (Checkbox)	Upload Photo
Upload Photo Upload Portfolio (showcase skills) CV upload (showcase work experience) Account Type (either ST, SU or both)	Account Type (either ST, SU or both)

The template for creating a profile includes the following:

- ST name, surname and information about them (a short introduction)

ST adds a skill that they are willing to showcase and teach; Student Tutor indicates their skill level using the scale provided (student, intermediate, professional); Tutor saves times of the day or week that they are available for teaching to their profile; ST can decide to also become a SU by changing the profile information settings by also checking the SU box. SU is a subset of ST; The option to upload a photo; Provide a place to upload a CV; Links to portfolios can be saved to ST profile.

TDA Tutor and Student Dashboard

The dashboard is what the SU and ST will see when they log in to TDA. Relevant menu items and information will be available to the corresponding logged in user. These include:

Display Dashboard; Show User Profile in menu bar; Display recommended Tutors for Students; Display recommended Students for tutors; Show bookmarks and quizzes navigation for SU; Show CV upload navigation for ST.

The Dashboard content will be based on the User type (if the user is ST or SU) and the backend part of TDA will have the two different template generators based on user type. They will be very similar- so to make it easier, some elements will be kept the same and re-used for both ST and SU like the Profile navigation for example. The other content that is specific to the user will then be added to the generic template based on who is logged in. The UI will be uncluttered and simple to use to make sure the TDA stays on brand (as an app to aid teaching and learning).

❖ Search component

When the SU (Student Upskill) wants to search for tutors, they will use a filter system to specify what they are looking for.

Tasks that the SU would be able to do include:

Display filter options; modify filter parameters through GUI interaction (to minimize typing errors); see the results for the filter function; bookmark tutors by clicking on the bookmark icon of the corresponding tutor; be able to view bookmarked tutors again by navigating to SU's favourite tutors (bookmark) section from the SU's dashboard.

Integration: The search filters component uses the Profile information of the student profile section of the main project (TDA). The Search Bookmark function for SU also integrates with the student profile section.

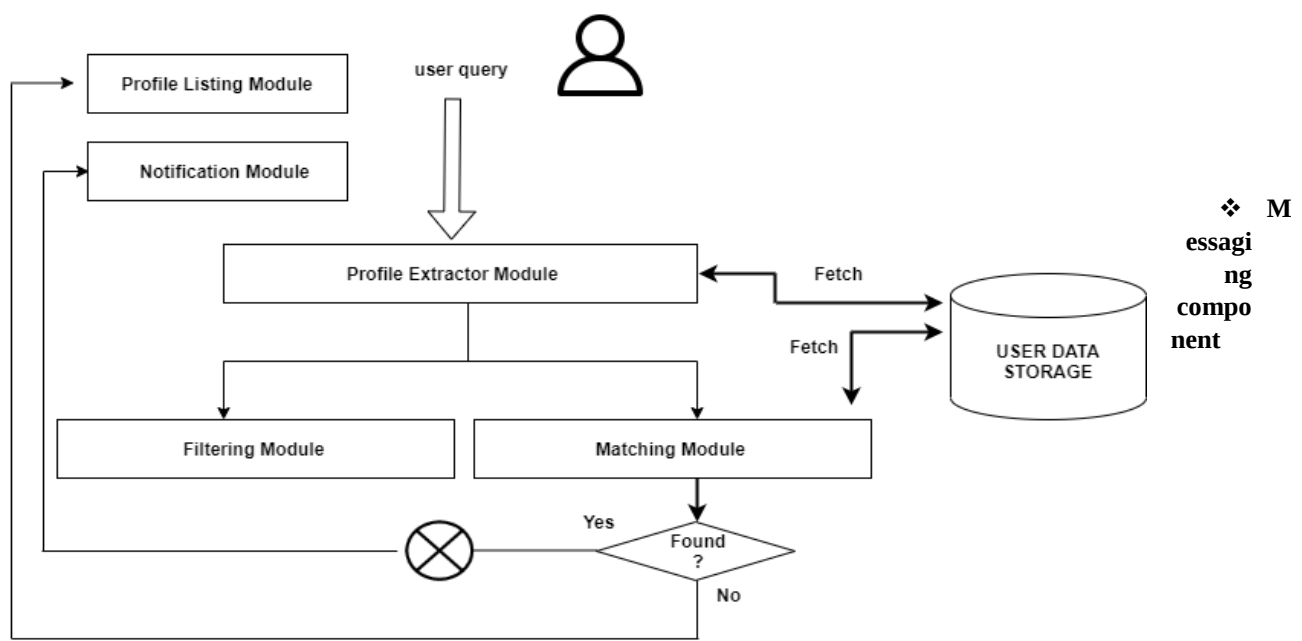
❖ Bookmark component

The purpose of having bookmarks is so that students can easily find tutors that they match with or that they found in searches. The Bookmark feature will allow students and tutors to 'bookmark' other students and tutors, essentially adding them to favourites list where they can find them instantly later.

By having the View update in response to the users selected menu items and search filters, the TDA will help the SU find suitable ST. Event triggers for mouse clicks on items on the User Interface will enable these functions to work. Queries to the database layer will help store, update and delete the favourite Student Tutor lists.

❖ Recommendation component

The purpose is to recommend a list of relevant tutors to a logged in student based on the information provided by the student while creating their profile. This is an automated filtering system which recommends tutors to a student based on the courses saved in their profile.

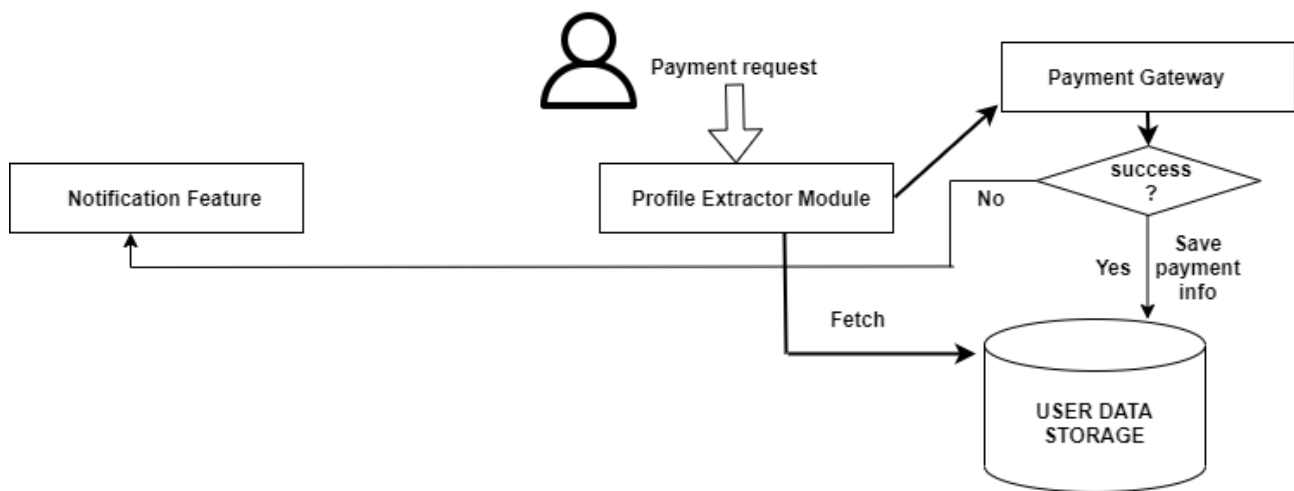


The messaging component serves as direct contact methods between students and tutors. By sending messages to each other the student and tutors can arrange suitable tutor times and other study related activities. If they wish to phone or email then this information would be provided on the respective profiles so that they can communicate after a match has been made on the TDA.

❖ Payment component

Its purpose is to give the tutor an overview of all payments received from other students. The tutor can submit a withdrawal request to his bank account from the wallet.

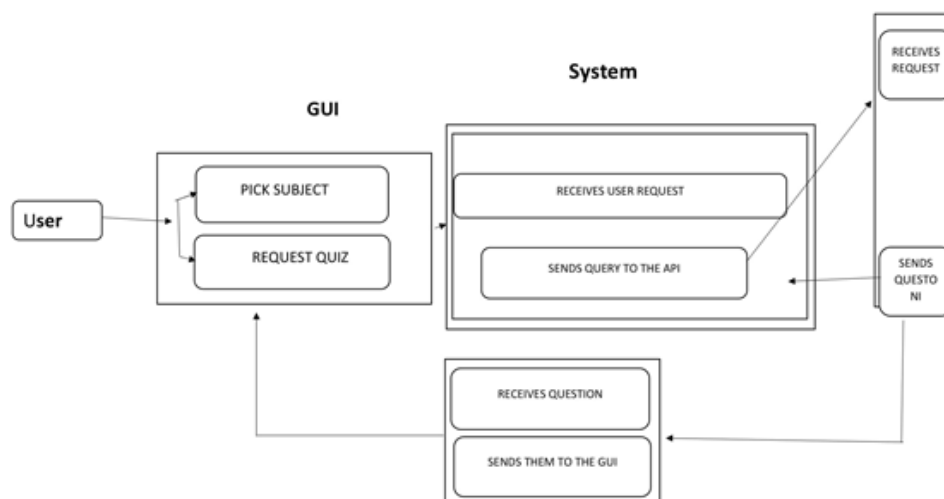
- Payment Collection Simulation: This feature is a simulation of a debit/credit card payment. Here it is a question of developing a checkout form which asks for debit/credit card information and proceeds with the payment.



❖ Self-Assessment component

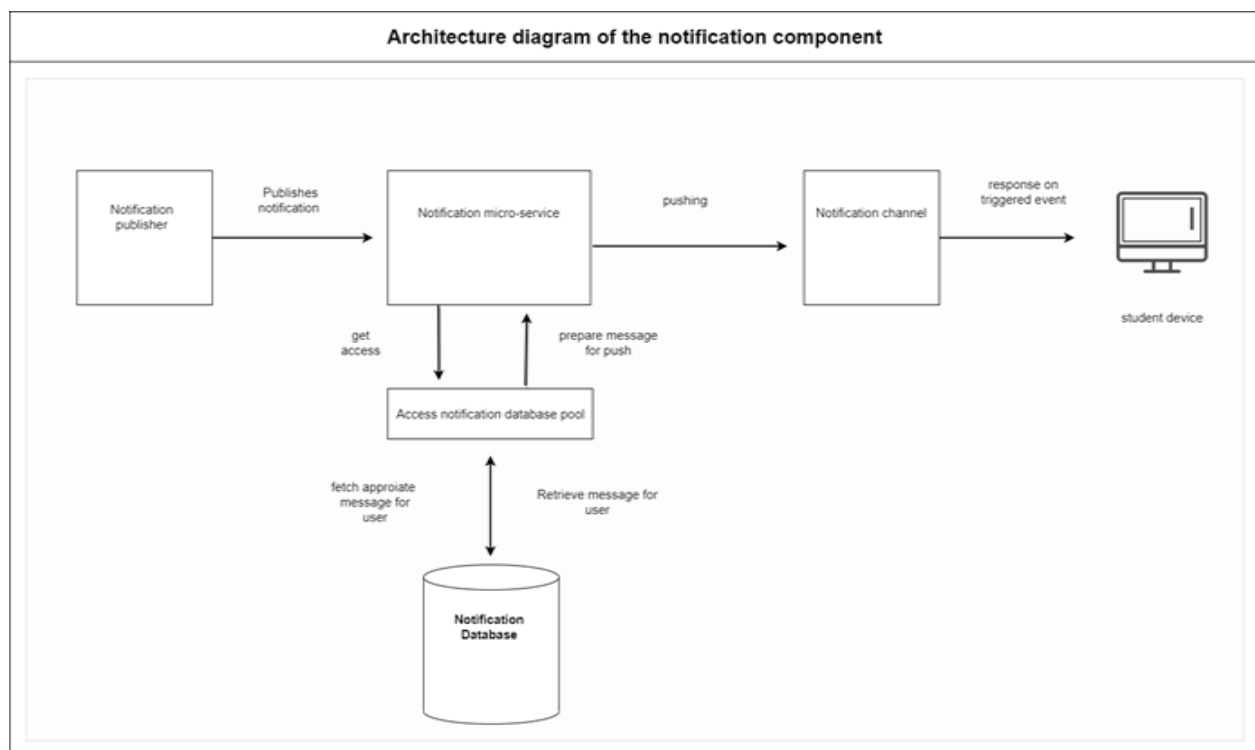
The self-assessment component main purpose is to help users assess themselves on different topics of their choosing.

- For administration, the administrator can log into the system and create topics, group questions under a specific topic. e.g., JAVA TDD (Test Driven Development) quiz, Laravel Fundamentals quiz, Java conditions and loops quiz and make the quizzes available for the students.
- for each question, Admin must set:
 - The topic related to the question
 - Question text
 - Options 1, 2, 3, 4, 5
 - Correct answer
 - Snippet code (optional)
 - answer explanation (optional)
 - Link for more info (optional)
- For students they can choose a topic where they are struggling or a topic where they want to test their knowledge and get assessed.



❖ Notification component

The purpose of this section of the application is to notify either the upskill or the tutor-student with messages relating to the completion of events stored in the notification database. Additionally, the notification component consists of sending messages to users across a variety of delivery channels, including email, SMS, PUSH API, etc. This will take place when via what they have registered with for instance once they have finished registering an account a message stating “We would like to congratulate you for the successful creation of your account with Talent Directory Application.” There is a list of events like payment and availability of tutor just to name a few which will send a notification based on the event that took place successfully or not.



The above is a representation of how the notifications will be handled in the Talent Directory Application. This component will be the backbone of the system since it will accommodate the point where the system can communicate through it to pass appropriate messages to end-users for clarity during or after an activity is performed within the application. For example, a pop-up message appears on the end-user’s screen saying, “Welcome to Talent Directory Application, your registration process was successful.”

Firstly, the notification publisher will publish a notification to the notification micro-service, it will try get access to the notification database pool to fetch the appropriate message for the user and retrieve the message from the database. Secondly, this message will be prepared for push and sent through the notification channel to the end-user’s screen.

❖ **Tutor Rating component**

The tutor rating system is the 5-star rating system our application will be using to display how well the tutors do their jobs and how confident students feel after a session with that tutor. There will also be a comment box for the students to add any additional comments on their experience with their various tutors when rating them.

The Tutor rating system will allow students to rate their experiences with their various tutors allowing others to see whether this tutor is good at what he or she offers or not. Students will be asked to rate tutors after every session has taken place and will allow them to add comments as well.

1. SYSTEM IMPLEMENTATION

The functionalities described previously will be deployed in a web environment with the following technical characteristics:

- Apache Server 2
- MySQL Database Management System 5.1 +
- PHP

4.10Registration component

The implementation of this authentication system in the Laravel framework concerns the Sign-up and Login functionalities. When a user is not yet logged in, he/she sees a login page where they can log in if they have an account in the system, or otherwise a link to create an account is offered.

The logic of account creation is provided by the User Controller located under app / Http / Controllers / Auth / inside this package we find 2 classes: LoginController and RegisterController. When the user clicks on login from the home page, he/she is redirected to the LoginController, specifically to the showLoginForm () method which displays the login form.

Furthermore, by clicking on Sign up from the login form, the user is redirected to the RegisterController within the showRegistrationForm method. The showRegistrationForm method retrieves all the fields of the registration form, retrieves data filled in, calls the User Model class, Set attributes, and save into the database.

Lastly, the login is managed by a middleware called Verified, which ensures that only unsecured pages are accessible by guests.

4.11Profile component

Creating Profile Edit Feature

The first time the user registers into the application, the system asks for some basic information such as name, email and password. The profile component allows the user to enter additional information for the user's use of the application. In our case, we need the user to choose their role in the application.

Two choices are offered: Tutor and student.

Apart from the role, the user must choose courses that interest him as a tutor or student. The courses saved will be useful for research and recommendation. The profile component also allows the user to upload his profile image, his CV, and to enter his contact number.

Procedure on the Front-end

- ☐ Profile Edit Page is accessible via the dashboard once the user is logged in.
- ☐ Then the profile edit form appears as soon as the user clicks on Profile
- ☐ The user then fills in his profile and submits the form

4.12Search component

Creating search/filter feature UI and Filter Feature implementation that allows the user to search for tutors based on their skills. It can be programming, Networking or Multimedia. The component only searches for instructors that match the applied filter. Students are excluded from the search and the user who does the research does not see his own profile even if he/she is a tutor.

Implementation for Backend:

The main method that does the search is located inside the PagesController class.

The path is: app/ Http/ Controllers/ PagesController.Php.

The method is called doSearchTutor.

The SQL query searches for all for all users who have created a profile where the role_id is 2 (Instructor role id) and the Courses must be of the category sent by the user during the search/filter operation.

And lastly, the search /filter feature allows a user to login or sign up their details, when done the user will be able to select which Course, they would like to find the Tutor for, given the options: Programming, Networking or Multimedia.

4.13Bookmark component

This component is designed to allow the student conducting the search to create a sort of short list that he or she can analyse later. When a student uses the search / filter component to filter tutors, a list of tutors is displayed, with the option for the user to bookmark some instructors. The user must be logged in order to save a tutor to their bookmarks; otherwise, the tutor will not be saved. Each thumbnail in the filter results has a small heart icon next to it that the user can click to save the profile to their bookmarks.

The code that saves the bookmarks is implemented in the User Controller (app / Http / controllers / User / UserController). There is a method within this class called jsonSaveBookmark. This method is called using Ajax to avoid refreshing the page.

4.14Recommendation component

Information flow

When a user is logged into the app, the recommendation component automatically read the student's profile, especially the courses the student is interested in.

The recommendation component then browses the profiles of the tutors who provide one or more courses saved in the student's profile.

If there are any profiles available, the component sends a ResultSet (Collection) to the home page where a loop has been implemented to extract and display those tutor's profiles.

4.15Wallet component

Information flow

A tutor can ask a student to make a payment in the app before he starts with the tutoring session. To make a payment, a student does the following:

- Visits the tutor's profile
- Clicks on make a payment
- Enters his bank details and validates the payment via the payment gateway.

The system sends a notification to the tutor and the student to confirm the payment. The tutor then receives a notification that a payment has been made, and connects to his account to view the transaction.

4.16 Self-Assessment component

- For administration, the administrator can log into the system and create topics, group questions under a specific topic. e.g., JAVA TDD (Test Driven Development) quiz, Laravel Fundamentals quiz, Java conditions and loops quiz and make the quizzes available for the students.
- for each question, Admin must set:
 - The topic related to the question
 - Question text
 - Options 1, 2, 3, 4, 5
 - Correct answer
 - Snippet code (optional)
 - answer explanation (optional)
 - Link for more info (optional)
- For students they can choose a topic where they are struggling or a topic where they want to test their knowledge and get assessed.

4.17 Notification component

The implementation of our Talent Directory Application notifications is short, informational messages that notify users of something that occurred in our application. For example, once a student made a payment to a tutor, we might send an "Invoice Paid" notification to both tutor and student via the email and SMS channels. Additionally, for this to occur the notification has to be triggered and it will be triggered once an event occurs, whether its successful or not.

4.18 Tutor Rating component

The tutor rating system is a 5-star rating system our application will be using to display how well the tutors do their jobs and how confident students feel after a session with that tutor. There will also be a comment box for the students to add any additional comments on their experience with their various tutors when rating them, which will be visible to other students. That way other students will not have to think about whether this tutor will work for them as they will be able to determine before booking a session whether this tutor is worth it or not. It will also help tutors determine whether their teaching methods or styles are effective with students based on the reviews they receive.

5. CONCLUSION

The artifact (TDA) would solve the problem of lack of student tutoring very well if used widely by the CPUT community. It would provide students with many opportunities to help each other understand the content and hone their IT skills.

6. REFERENCE



Anonymous student survey

Collect student's concerns regarding technical skill and application of course knowledge

Is your field of study one of the following: programming/multimedia/networking

☐ Yes

☐ No

☐ Other: _____

Do you have a technical or practical element in your studies?

☐ Yes

☐ No

☐ Other: _____

Do you feel confident and ready to work in your chosen field's industry?

1 2 3 4 5

Not confident ☐ ☐ ☐ ☐ ☐ Very confident

Has the pandemic had a positive or negative impact on your technical application of your studies?

1 2 3 4 5

Negative - I feel the lack of contact class has made me lacking in my technical application ☐ ☐ ☐ ☐ ☐ Positive - it has helped me apply my skill better

Do you think that getting help from other more skilled students would improve your technical skill?

☐ Yes

☐ No

☐ Other: _____

If an application could help you get in touch with more skilled students - would you use that?

☐ Yes

☐ No

☐ Other: _____

Rate the amount of value such an application would add to your student life:

1 2 3 4 5

Not much - I prefer things the way they are now ☐ ☐ ☐ ☐ ☐ It would help me be more prepared for the working environment after my studies

If you could earn extra income by teaching students how to apply their knowledge in a technical field - would you want to do that?

☐ Yes

☐ No

☐ Other: _____

Submit